Coding Assessment: QuickDesk (AI-Assisted Helpdesk)

Build **QuickDesk**, a small internal helpdesk app that uses an LLM to assist support agents.

Imagine an internal company tool where employees raise support tickets ("My VPN is not connecting", "Need access to the finance drive", "Laptop battery is dying") and a small team of support agents responds to them. The AI does NOT replace the agent. It helps the agent move faster.

Core scope (must build)

Auth and roles

1. JWT-based authentication.
2. Two roles: **employee** and **agent**.
    - Employees can submit tickets and view only their own tickets.
    - Agents can see all tickets, reply, and resolve them.
3. Role enforcement must be on the backend, not just hidden buttons on the frontend. We will test this.
4. Passwords hashed with bcrypt or argon2. Plaintext is an instant fail.

Ticket submission (employee side) 5. A simple form: title, description, and optional attachment field (just store filename, no real upload needed). 6. When a ticket is submitted, the backend uses an LLM to:

- Suggest a category (IT, HR, Finance, Admin, Other)
- Suggest a priority (Low, Medium, High)
- Both values are stored on the ticket but marked as "AI-suggested" (the agent can override later).

7. Employees should see a "My Tickets" view with status of each of their tickets.

Agent dashboard 8. List view of all tickets with filters (status, category, priority) and a search box on title. 9. Ticket detail view that shows:

- The original ticket and the employee who raised it
- The AI-suggested category and priority (agent can override either)
- An **AI-drafted reply** generated using RAG over a small knowledge base (see below)
- An editor where the agent can edit the AI draft and click "Send Reply"
- **Citations:** show which knowledge base article(s) the AI pulled from

10. Once "Send Reply" is clicked, the ticket moves to "Resolved" and the final reply is stored. The AI draft and the final reply are both kept so we can later compare what the AI suggested vs what was actually sent.
11. **Override audit log:** whenever an agent changes the AI-suggested category or priority, log it (who, when, from, to). Visible on the ticket detail page.

Real-time updates 12. When an employee submits a new ticket, all agents currently viewing the dashboard should see it appear without refreshing. 13. When an agent replies, the employee viewing "My Tickets" should see the status flip to "Resolved" without refreshing. 14. Use Socket.io, native WebSockets, or Server-Sent Events. Document the choice and why in the README.

Knowledge base for RAG 15. Seed the system with 5 to 10 small markdown "knowledge base" articles (sample topics: VPN setup, password reset policy, leave application process, expense reimbursement, laptop request). Write them yourself, keep them short, 100 to 300 words each. 16. The AI's drafted reply must be grounded in these articles. If no article is relevant, the AI should say so rather than hallucinate.

Basic metrics page (agent-only) 17. A simple dashboard route showing:

- Total tickets by status (Open / Resolved)
- Tickets by category (count)
- Median resolution time
- How often agents overrode the AI's suggested category (a percentage)

That is the core scope. Polish over scope. A clean, working implementation of the above beats a feature-bloated half-broken one.

Stretch goals (optional, only if you finish core comfortably)

Pick at most one or two. Do NOT attempt all.

- **Dockerize:** A working `docker-compose.yml` that spins up backend + frontend + Mongo with a single command.
- **Email notifications:** When an agent replies, send a (mock or real) email notification to the employee. Logging the email body to console is acceptable.
- **AI confidence score:** Have the LLM return a confidence score with its category suggestion and surface it in the UI.
- **Rate limiting:** Limit ticket submission to N per hour per employee.
- **Tests:** A handful of meaningful tests on the most critical paths (auth, RAG endpoint, role enforcement).

We would rather see a polished core than a half-finished pile of stretch goals.

Constraints and choices

Tech stack

- **Frontend:** React or Next.js (your choice). TypeScript preferred but not required.
- **Backend:** Nest.js, OR Python + FastAPI. Pick one.
- **Database:** Postgres.
- **LLM:** Any provider you can access on a free tier. OpenAI, Gemini, Groq (free), or a free Hugging Face inference endpoint. If you cannot get a paid key, use a free-tier model. Document this clearly in the README.
- **LangChain:** Use it for the RAG pipeline. We want to see you can wire up a real RAG flow, even a simple one. In-memory vector store is fine (Chroma, FAISS, or LangChain MemoryVectorStore). No need for Pinecone.
- **Auth:** JWT. Store passwords using bcrypt (or argon2). Plaintext passwords are an instant fail.
- **Styling:** Anything. Tailwind, plain CSS, MUI. Not graded heavily, but it should look intentional, not broken.

Things we will not accept

- A repo dumped in one massive commit at the end
- A README that is obviously written entirely by an AI with no specific decisions or numbers from your actual implementation
- Hard-coded API keys committed to git (use `.env.example`)
- Plaintext passwords
- A copy of a public boilerplate with the brief bolted on top

Deliverables

1. GitHub repository

- Public repo, named something like `quickdesk` (your choice).
- Working `npm install` (or `pip install -r requirements.txt`) + clear run instructions.
- `.env.example` with all required env variables listed but no real keys.
- Seed script that creates at least one sample agent user, one sample employee user, and loads the knowledge base articles.

2. README.md (this matters a lot)

Cover these sections:

- **What this is** (one paragraph)
- **How to run it locally** (step by step, must work on a fresh clone)
- **Architecture diagram** (text-based or image, your call)
- **API endpoints** (table: method, path, purpose, auth required)
- **Decisions and Tradeoffs** (see prompt below)
- **What I would do with more time**
- **Known issues / limitations**

For "Decisions and Tradeoffs", answer at least these in your own words:

a) Why did you pick this frontend framework (React vs Next.js)? b) How did you structure the RAG pipeline? Chunk size, embedding model, retriever, prompt? c) How did you handle the case where the LLM returns a category that does not match your allowed list? d) Where did you store the JWT on the client, and why? e) How did you enforce role-based access on the backend? What stops an employee from hitting an agent-only endpoint by guessing the URL? f) Why did you pick Socket.io / WebSockets / SSE for real-time? What is the failure mode if the socket disconnects mid-session? g) What is the worst failure mode in your system today, and what would you do to address it? h) Where did AI tools help you most? Where did they hurt or mislead you?

3. Demo Video (3 to 5 minutes)

- Show: submitting a ticket, the AI suggestion appearing, the agent replying with the AI draft.
- Talk through the architecture in 1 to 2 minutes.
- Keep it casual. We are not grading presentation polish.

Quick FAQ

Q: What if I cannot get an LLM API key? A: Use Groq (free tier, fast). If you absolutely cannot, mock the LLM call with a function that returns realistic-looking output, and document this in the README. We will not penalize you for provider access, but we WILL test your code path.

Q: Can I use a boilerplate / starter template? A: Yes, but disclose it in the README. Avoid templates that already include helpdesk-like features.

Q: Can I skip JWT and use sessions? A: Yes, but explain the tradeoff in the README.

Q: How polished must the UI be? A: It should look like you cared. It does not need to look like a designer touched it.

Q: What if I run out of time? A: Submit what you have, with a clear "what is missing" section in the README. Honesty scores higher than overreach.

Q: How important is the Loom? A: Important. It is the cheapest way for us to see how you think. Skipping it is a missed signal.

Good luck. We are looking forward to seeing what you ship.